JavaScript Equality Operators

== vs === -- know the difference, avoid the bugs

JavaScript gives you two ways to check if things are equal, and picking the wrong one is one of the most common sources of bugs. The difference boils down to one question: should JavaScript try to convert the types before comparing, or should it compare them exactly as they are?

== (Double equals): Loose equality -- compares values only, converts types if needed.
=== (Triple equals): Strict equality -- compares both value AND type, no conversions.

Loose Equality (==) -- the forgiving one

When you use ==, JavaScript performs something called **type coercion**. It looks at both sides, and if the types don't match, it quietly converts one (or both) values to a common type before comparing. This can be handy sometimes, but it also leads to some genuinely surprising results.

```
let n1 = 10;        // number
let n2 = "10";      // string

console.log(n1 == n2);    // true
// JavaScript converts "10" (string) to 10 (number), then compares
```

Here's where it gets weird:

```
console.log(true == 1);     // true   (true becomes 1)
console.log(false == 0);    // true   (false becomes 0)
console.log("" == 0);       // true   (empty string becomes 0)
console.log([] == 0);       // true   (empty array becomes 0)
```

Why is this risky? Because an empty string, false, 0, and an empty array all look "equal" to each other under ==. That's almost never what you actually mean in your code.

Strict Equality (===) -- the precise one

With ===, what you see is what you get. No conversions, no surprises. If the types don't match, it immediately returns false without trying to "help" you. This is almost always what you want.

```
let n1 = 10;        // number
let n2 = "10";      // string

console.log(n1 === n2);   // false
// Different types (number vs string) = not strictly equal. Period.
```

Same examples, but with strict equality:

```
console.log(true === 1);    // false  (boolean is not number)
console.log(false === 0);   // false  (boolean is not number)
console.log("" === 0);      // false  (string is not number)
console.log([] === 0);      // false  (object is not number)
```

Much more predictable. With ===, if the types don't match, the answer is always false. No guesswork, no hidden conversions.

Special Case: null vs undefined

This one comes up in interviews all the time, so it's worth understanding well. null and undefined are both "empty" values in JavaScript, but they're technically different types with a quirky relationship.

```
console.log(null == undefined);     // true
// Special rule: JS treats them as loosely equal to each other

console.log(null === undefined);    // false
// Different types, so strict equality says no

console.log(typeof(null));          // "object"  <-- a famous JS bug!
console.log(typeof(undefined));     // "undefined"
```

The fact that typeof null returns "object" is a well-known bug that dates back to the very first version of JavaScript. It was never fixed because too much existing code depends on it. null is not actually an object -- it's its own primitive type.

Practical example:

```
if (null === undefined) {
    console.log('hi click on the element');
} else {
    console.log('close browser');   // This runs
}

// Since null and undefined have different types,
// strict equality returns false, so we hit the else block.
```

Side-by-Side Comparison

Here's the full picture. Look at how differently == and === treat the same comparisons:

Comparison	== (Loose)	=== (Strict)	What Happens
`10 == "10"`	true	false	String converted to number with ==
`true == 1`	true	false	Boolean converted to number with ==
`false == 0`	true	false	Boolean converted to number with ==
`"" == 0`	true	false	Empty string converted to 0 with ==
`[] == 0`	true	false	Empty array converted to 0 with ==
`null == undefined`	true	false	Special JS rule for == only

Best Practices

- **Default to === (strict equality).** It's more predictable, catches type mismatches early, and prevents subtle bugs that are hard to track down.

- **Use == only when you deliberately want type coercion.** In practice, this is rare. The most common legitimate use is checking for null or undefined in one shot: `if (value == null)` catches both.

- **Watch out for the null/undefined quirk.** Know that null == undefined is true, but null === undefined is false. This is an interview favorite.

- **Remember that typeof null returns "object".** It's a historical bug. Don't let it confuse you into thinking null is an object.

Common Pitfalls to Avoid

- Unexpected type conversions with == -- empty strings, false, 0, and empty arrays can all appear "equal" to each other.

- Assuming null is an object because typeof says so. It's not. It's a primitive.

- Using == by habit instead of by choice. Most linters will flag this for good reason.

Memory tip to keep them straight:

== means "equal enough" -- loose, forgiving, tries to make things work.
=== means "exactly equal" -- strict, precise, no shortcuts.

When in doubt, go with ===. Your future self will thank you.

Understanding == vs === is one of those fundamentals that separates someone who writes JavaScript from someone who truly understands it. Master this, and you'll sidestep a whole category of bugs.